StockScan Desktop — Code Signing Guide

This guide explains how to get a code signing certificate and configure it for building signed `.exe` installers.

Why Code Sign?

Without code signing:
- Windows SmartScreen shows “Windows protected your PC” warning
- Some antivirus software may flag the installer
- Users see “Unknown Publisher” in the installer

With code signing:
- Users see “StockScan Ltd” as the verified publisher
- No SmartScreen warnings (after building reputation)
- Professional and trustworthy installation experience

Step 1: Get a Code Signing Certificate

Option A: Standard Code Signing Certificate (~£70-200/year)

Best for: Getting started, small businesses

Providers:
- **Sectigo (Comodo)**: https://sectigo.com/code-signing-certificates
- **DigiCert**: https://www.digicert.com/signing/code-signing-certificates
- **GlobalSign**: https://www.globalsign.com/en/code-signing-certificate
- **SSL.com**: https://www.ssl.com/certificates/code-signing/

> **Note**: As of June 2023, certificate authorities require hardware tokens (USB) or cloud HSM for code signing. “Standard” OV certificates are now delivered on hardware tokens.

Option B: EV Code Signing Certificate (~£300-500/year)

Best for: Immediate SmartScreen trust, enterprise deployments

- Provides **immediate** SmartScreen reputation (no warming period)
- Displayed as “Extended Validation” in certificate details
- Same providers as above, but select the EV option

Recommended: SSL.com eSigner (Cloud-Based)

SSL.com offers cloud-based signing via their eSigner service, which works well with CI/CD:
- No physical USB token needed
- Works in GitHub Actions without special hardware
- Pricing: ~$200-350/year for OV, ~$300-500/year for EV
- Setup guide: https://www.ssl.com/how-to/automate-ev-code-signing-with-signtool-or-esigner/

Step 2: Export Your Certificate

Once you receive your certificate, export it as a `.pfx` (PKCS#12) file:

From a USB Token (SafeNet/YubiKey):

1. Install the token drivers
2. Open the certificate management tool
3. Export as `.pfx` with a strong password

From Windows Certificate Store:

1. Open `certmgr.msc`
2. Find your code signing certificate
3. Right-click → All Tasks → Export
4. Select “Yes, export the private key”
5. Choose PKCS #12 (.PFX) format
6. Set a strong password
7. Save the file

Step 3: Configure GitHub Actions

3a. Base64-encode your .pfx file:

```
# On Linux/macOS:
base64 -i your-certificate.pfx -o certificate-base64.txt

# On Windows (PowerShell):
[Convert]::ToBase64String([IO.File]::ReadAllBytes("your-certificate.pfx")) | Out-File
certificate-base64.txt
```

3b. Add GitHub Secrets:

Go to your GitHub repo → Settings → Secrets and variables → Actions → New repository secret:

Secret Name	Value
`WIN_CSC_LINK`	The entire contents of `certificate-base64.txt`
`WIN_CSC_KEY_PASSWORD`	The password you set when exporting the .pfx

3c. Trigger a build:

Option 1 — Push a version tag:

```
git tag v1.0.0
git push origin v1.0.0
```

Option 2 — Manual trigger:

1. Go to Actions tab in GitHub
2. Select “Build Desktop App” workflow
3. Click “Run workflow”
4. Optionally set `skip_signing: true` for testing

Step 4: Build Locally (Optional)

Signed build (certificate on your machine):

```
cd electron-app
npm install

# Set certificate environment variables:
export WIN_CSC_LINK="path/to/your-certificate.pfx"
export WIN_CSC_KEY_PASSWORD="your-pfx-password"

npm run build:win
```

Unsigned build (for testing):

```
npm run build:win:unsigned
```

The installer will be in the `dist/` folder: `StockScan-Setup-1.0.0.exe`

Step 5: Distribute

After the GitHub Actions build completes:
1. Go to your repo's Releases page
2. The `.exe` will be attached to the release
3. Copy the download URL and update the StockScan download page

SmartScreen Reputation

Even with a signed installer:
- **OV certificates**: SmartScreen may still warn for the first ~500-1000 downloads
- **EV certificates**: Immediate trust, no warning period
- Each new version resets the counter for OV certificates

Troubleshooting

"Windows protected your PC" still showing

- This is normal for new OV certificates — reputation builds over time
- Consider upgrading to an EV certificate for immediate trust
- Submit your app to Microsoft: https://www.microsoft.com/en-us/wdsi/filesubmission

Build fails with signing error

- Verify `WIN_CSC_LINK` is properly base64-encoded (no line breaks)
- Verify `WIN_CSC_KEY_PASSWORD` is correct
- Check certificate hasn't expired

"Certificate not valid for code signing"

- Ensure you purchased a **Code Signing** certificate (not SSL/TLS)
- Check the certificate's Enhanced Key Usage includes "Code Signing"